

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 3522 PROJE RAPORU

Projenin Başlığı

Gerçek Zamanlı Veri Akışı ve İşleme

Öğrencinin Adı Soyadı

Erkim Berk Ünsal

Öğrenci Numarası

23291218

Öğrenci Adı Soyadı	Erkim Berk Ünsal
Öğrenci Numarası	23291218
Ders Adı ve Kodu	BLM3522 – Bulut Bilişim ve Uygulamaları
Proje No	Proje 2
Backend Teknolojisi	Python 3.12 + paho-mqtt 1.6.1
Protokol	MQTT over TLS 1.2
Veritabanı	AWS DynamoDB
Bulut Platformu	AWS (IoT Core, Lambda, DynamoDB, CloudWatch)
Tarih	6 Nisan 2026
Github Repo Linki	https://github.com/erkimberk/BLM3522
Video Linki	https://youtu.be/w4XeVd347yE

1. GİRİŞ

Bu proje, AWS bulut altyapısı kullanılarak gerçek zamanlı IoT veri akışı ve işleme sisteminin tasarımı ve uygulamasını kapsamaktadır. Proje kapsamında Python ile yazılmış bir sensör simülatörü, MQTT protokolü aracılığıyla AWS IoT Core'a bağlanmakta; gelen veriler AWS Lambda tarafından işlenerek anomali tespiti yapılmakta ve sonuçlar DynamoDB'de depolanmaktadır.

Sistem gerçek bir IoT ortamını simüle etmekte; sıcaklık ve nem verileri üretilmekte, bu veriler TLS 1.2 şifreli bağlantıyla buluta iletilmektedir. Tüm bileşenler AWS'nin eu-north-1 bölgesinde çalışmaktadır.

1.1 Projenin Amacı

- MQTT protokolüyle IoT cihaz verisi üretmek ve buluta iletmek
- AWS IoT Core üzerinde güvenli cihaz yönetimi gerçekleştirmek
- Gerçek zamanlı veri işleme için AWS Lambda kullanmak
- İşlenen verileri DynamoDB'de kalıcı olarak saklamak
- Anomali tespiti ve uyarı sistemi kurmak
- CloudWatch ile sistem izleme sağlamak

2. SİSTEM MİMARİSİ

Proje beş temel katmandan oluşan olay güdümlü (event-driven) bir mimari üzerine inşa edilmiştir. Her katman birbirini tetikleyerek uçtan uca gerçek zamanlı veri akışını sağlar.

2.1 Mimari Bileşenler

Bileşen	AWS Servisi	Bölge	Açıklama
Simülatör	Python / paho-mqtt	Yerel	IoT sensör verisi üretir
MQTT Broker	AWS IoT Core	eu-north-1	Cihaz bağlantısı ve mesaj yönlendirme
Kural Motoru	AWS IoT Rule	eu-north-1	SQL ile mesaj filtreleme ve Lambda tetikleme
İşlemci	AWS Lambda	eu-north-1	Doğrulama, anomali tespiti ve dönüşüm
Veritabanı	AWS DynamoDB	eu-north-1	İşlenmiş verilerin kalıcı olarak saklanması
İzleme	AWS CloudWatch	eu-north-1	Log yönetimi ve sistem izleme

2.2 Veri Akış Diyagramı

Sistem bileşenleri şu sırayla çalışmaktadır:

```
[ Sensör Simülatörü (Python) ]
    | MQTT over TLS 1.2 (Port 8883)
    ↓
[ AWS IoT Core ] → Cihaz: iot-sensor-001
    | Topic: sensors/iot-sensor-001/data
    ↓
[ AWS IoT Rule ] → SQL: SELECT * FROM 'sensors/+/data'
    | Action: Lambda tetikleme
    ↓
[ AWS Lambda ] → Doğrulama + Anomali Tespiti + Dönüşüm
    | PutItem
    ↓
[ AWS DynamoDB ] → Tablo: iot-sensor-data
    |
    ↓
[ AWS CloudWatch ] → Log: /aws/lambda/iot-sensor-processor-new
```

2.3 Güvenlik Mimarisi

Sistem güvenliği birden fazla katmanda sağlanmaktadır:

- TLS 1.2: MQTT bağlantısı uçtan uca şifreli
- X.509 Sertifika: Cihaz kimlik doğrulama için AWS tarafından imzalı sertifika
- IAM Rolü: Lambda'ya yalnızca gerekli DynamoDB ve CloudWatch izinleri
- IoT Policy: Cihazın yalnızca kendi topic'ine erişimi

3. KULLANILAN TEKNOLOJİLER

3.1 Protokol: MQTT

MQTT (Message Queuing Telemetry Transport), IoT uygulamaları için tasarlanmış hafif bir mesajlaşma protokolüdür. Düşük bant genişliği ve kısıtlı cihazlar için idealdir. Bu projede

MQTT v3.1.1 sürümü kullanılmış, QoS 1 (en az bir kez iletim garantisi) seviyesi tercih edilmiştir.

3.2 AWS IoT Core

AWS IoT Core, milyonlarca IoT cihazını güvenli şekilde bağlamaya ve yönetmeye olanak tanıyan tam yönetilen bir bulut servsidir. X.509 sertifika tabanlı kimlik doğrulama ve şifreli iletişim destekler. Proje konfigürasyonu:

- Endpoint: avd01ikm8qx2k-ats.iot.eu-north-1.amazonaws.com:8883
- Cihaz Adı: iot-sensor-001
- Topic: sensors/iot-sensor-001/data
- Protokol: MQTT over TLS 1.2

3.3 AWS Lambda

AWS Lambda, sunucu yönetimi gerektirmeyen (serverless) bir bilgi işlem servsidir. IoT Rule tarafından tetiklenen Lambda fonksiyonu Python 3.12 çalışma ortamında çalışmakta; veri doğrulama, anomali tespiti ve DynamoDB yazma işlemlerini gerçekleştirmektedir.

3.4 AWS DynamoDB

DynamoDB, tam yönetilen bir NoSQL veritabanıdır. Milisaniye düzeyinde gecikme süresi ve otomatik ölçekleme özellikleri ile gerçek zamanlı veri depolama için uygundur. Tabloda Bölüm Anahtarı olarak sensor_id, Sıralama Anahtarı olarak timestamp kullanılmaktadır.

3.5 Kullanılan Python Kütüphaneleri

Kütüphane	Sürüm	Kullanım Amacı
boto3	1.34.0	AWS servisleriyle iletişim (DynamoDB, vb.)
paho-mqtt	1.6.1	MQTT protokolü istemci kütüphanesi
python-dotenv	1.0.0	Ortam değişkenlerinin yönetimi
requests	2.31.0	HTTP istekleri
colorama	0.4.6	Terminal çıktısı renklendirme

4. KOD UYGULAMASI

4.1 IoT Sensör Simülatörü (sensor_sim.py)

Sensör simülatörü, gerçek bir IoT cihazının davranışını taklit eden Python sınıfı (IoTSensorSimulator) içermektedir. MQTT bağlantısı TLS 1.2 şifrelemesiyle kurulmakta ve veriler belirli aralıklarla yayınlanmaktadır.

MQTT Bağlantı Yapılandırması:

```
mqtt_client = mqtt.Client(client_id='iot-sensor-001',
                           protocol=mqtt.MQTTv311)

mqtt_client.tls_set(
    ca_certs='AmazonRootCA1.pem',
    certfile='*-certificate.pem.crt',
    keyfile='*-private.pem.key',
    cert_reqs=ssl.CERT_REQUIRED,
    tls_version=ssl.PROTOCOL_TLSv1_2
)

mqtt_client.connect(AWS_ENDPOINT, port=8883)
```

Sensör Verisi Üretimi:

```
def generate_sensor_data(self):
    return {
        'sensor_id': 'iot-sensor-001',
        'location': 'Lab-Odasi-1',
        'temperature': round(random.uniform(15, 35), 2),
        'humidity': round(random.uniform(30, 90), 2),
        'timestamp': datetime.utcnow().isoformat() + 'Z'
    }
```

MQTT Yayın (Publish) İşlemi:

```
topic = f'sensors/{self.sensor_id}/data'
payload = json.dumps(self.generate_sensor_data())
self.mqtt_client.publish(topic, payload, qos=1)
```

4.2 AWS Lambda Fonksiyonu (lambda_function.py)

Lambda fonksiyonu beş aşamalı işlem hattı uygulamaktadır: veri ayrıştırma, doğrulama, anomali tespiti, dönüşüm ve DynamoDB'ye yazma.

Ana İşleyici Fonksiyon:

```
def lambda_handler(event, context):
    # AŞAMA 1: Veri ayrıştırma
    sensor_id = event['sensor_id']
    temperature = float(event['temperature'])
    humidity = float(event['humidity'])
    timestamp = event['timestamp']
    location = event.get('location', 'Bilinmiyor')
```

```

# AŞAMA 2: Anomali tespiti
alert, severity, messages = analyze_data(temperature, humidity)

# AŞAMA 3: DynamoDB'ye kaydet
table.put_item(Item={
    'sensor_id'      : sensor_id,
    'timestamp'      : timestamp,
    'temperature'    : Decimal(str(temperature)),
    'humidity'       : Decimal(str(humidity)),
    'alert'          : alert,
    'alert_severity' : severity,
    'alert_messages' : messages
})

```

Anomali Tespit Mantığı:

```

def analyze_data(temperature, humidity):
    alert, severity, messages = False, 'INFO', []

    if temperature > 35 or temperature < 0:
        severity = 'CRITICAL'
        messages.append(f'Kritik sıcaklık: {temperature}C')
    elif temperature > 30 or temperature < 15:
        severity = 'WARNING'
        messages.append(f'Yuksek sıcaklık: {temperature}C')

    if humidity > 85 or humidity < 25:
        if severity != 'CRITICAL': severity = 'WARNING'
        messages.append(f'Anormal nem: {humidity}%')

    alert = len(messages) > 0
    return alert, severity, messages

```

Kritik Uygulama Detayı - Decimal Dönüşümü:

DynamoDB, Python'un float tipini kabul etmediğinden float değerlerin Decimal tipine dönüştürülmesi zorunludur:

```

# YANLIŞ: float doğrudan kullanılmaz
'temperature': 23.45          # Hata verir

# DOGRU: Decimal'e donusturmek gerekir
'temperature': Decimal(str(23.45))  # Çalışır

```

5. AWS YAPILANDIRMASI

5.1 AWS IoT Core Kurulumu

AWS IoT Core konsolundan iot-sensor-001 adlı IoT cihazı oluşturulmuş ve aşağıdaki bileşenler yapılandırılmıştır:

- Cihaz (Thing): iot-sensor-001
- X.509 Sertifika: AWS tarafından oluşturulmuş, cihaza bağlı
- IoT Policy: iot-sensor-policy (tüm iot:* eylemlere izin verir)
- Endpoint: avd01ikm8qx2k-ats.iot.eu-north-1.amazonaws.com

5.2 IoT Kural (Rule) Yapılandırması

```
Kural Adı      : iot_to_lambda_rule
Durum          : ENABLED
SQL Sorgusu    : SELECT * FROM 'sensors/+/data'
Eylem          : Lambda fonksiyonu tetikle
Fonksiyon      : iot-sensor-processor-new
```

5.3 Lambda IAM Yetkilendirmeleri

Lambda fonksiyonunun çalışması için iot-sensor-processor-role adlı IAM rolü oluşturulmuş ve aşağıdaki izinler verilmiştir:

```
{
  'Effect': 'Allow',
  'Action': [
    'dynamodb:PutItem',
    'dynamodb:GetItem',
    'dynamodb:Query'
  ],
  'Resource': 'arn:aws:dynamodb:eu-north-1:...:table/iot-sensor-data'
}
```

5.4 DynamoDB Tablo Şeması

Alan Adı	Tip	Açıklama
sensor_id	String	Bölüm Anahtarı (Partition Key)
timestamp	String	Sıralama Anahtarı (Sort Key) - ISO 8601
location	String	Sensör konumu
temperature	Number	Sıcaklık değeri (°C)
humidity	Number	Nem değeri (%)
received_at	String	Verinin alındığı zaman
processed_at	String	İşlenme zamanı
alert	Boolean	Uyarı bayrağı (True/False)
alert_severity	String	INFO / WARNING / CRITICAL
alert_messages	List	Uyarı mesajları dizisi

Alan Adı	Tip	Açıklama
analysis_version	String	Analitik sürümü

6. TEST VE SONUÇLAR

6.1 Test 1: Normal Veri Akışı

Normal sıcaklık (23.5°C) ve nem (65%) değerleriyle gönderilen verinin doğru şekilde işlendiği doğrulanmıştır.

```
Girdi : { temperature: 23.5, humidity: 65.0 }
Cikti : statusCode=200, alert=false, severity=INFO
DynamoDB: Kayıt oluşturuldu - alert: false
```

6.2 Test 2: Anomali Tespiti (Yüksek Sıcaklık)

Eşik değerini (30°C) aşan bir sıcaklık verisi gönderilmiş ve sistemin doğru uyarı ürettiği doğrulanmıştır.

```
Girdi : { temperature: 35.0, humidity: 65.0 }
Cikti : statusCode=200, alert=true, severity=WARNING
Mesaj : 'Yuksek sicaklik: 35.0C'
DynamoDB: Kayıt oluşturuldu - alert: true, alert_severity: WARNING
```

6.3 Test 3: Uçtan Uca Entegrasyon Testi

sensor_sim.py çalıştırılarak 11 MQTT mesajı AWS IoT Core'a iletilmiştir.

```
Sonuc:
- MQTT bağlantısı başarılı
- 11 mesaj yayınlandı (%100 iletim oranı)
- IoT Rule 11 mesajı Lambda'ya iletti
- Lambda 11 mesajı başarıyla işledi
- DynamoDB'de 11 kayıt oluşturuldu
- CloudWatch'ta tüm loglar görüntülendi
```

6.4 Performans Metrikleri

Metrik	Değer
MQTT Mesaj İletim Oranı	%100 (11/11)
Lambda İşlem Süresi	30 - 279 ms
Lambda Başarı Oranı	%100
DynamoDB Yazma Gecikmesi	< 50 ms
Veri Tazeliği (Uçtan Uca)	< 1 saniye
Sistem Çalışma Süresi	%100

7. GİT SÜRÜM KONTROLÜ

Proje boyunca Git ile düzenli sürüm kontrolü uygulanmıştır. Her anlamlı adım ayrı bir commit olarak kayıt altına alınmıştır.

#	Tarih	Commit Mesajı
1	03.04.2026	Proje klasör yapısı, .gitignore ve README.md taslağı oluşturuldu
2	03.04.2026	requirements.txt ile gerekli Python kütüphaneleri eklendi
3	03.04.2026	IoT sensör simülatörü yazıldı (AWS bağlantısız, sadece veri üretme)
4	03.04.2026	AWS endpoint ile sensor_sim.py güncellendi
5	03.04.2026	IoT Core → Lambda Rule tamamlandı ve Lambda fonksiyonu kodlandı
6	03.04.2026	DynamoDB tablosu ve Lambda IAM yetkilendirmeleri yapılandırıldı
7	03.04.2026	aws_config güncellendi

8. PROJE DOSYA YAPISI

```
Project2/
├── README.md                # Genel belgelendirme
├── README_TECHNICAL.md     # Teknik detay raporu
├── requirements.txt         # Python bağımlılıkları
├── .gitignore              # Sertifika dosyaları hariç tutuluyor
├── src/
│   ├── sensor_sim.py       # IoT Sensör Simülatörü
│   └── lambda_function.py  # AWS Lambda kodu
├── certificates/
│   ├── AmazonRootCA1.pem
│   ├── AmazonRootCA3.pem
│   ├── *-certificate.pem.crt
│   └── *-private.pem.key
└── reports/
    └── (Proje raporu ve belgeler)
```

9. SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında AWS bulut servisleri kullanılarak üretim kalitesinde gerçek zamanlı bir IoT veri akış sistemi başarıyla hayata geçirilmiştir. Proje aşağıdaki kazanımları sağlamıştır:

9.1 Teknik Kazanımlar

- IoT Veri Toplama: MQTT protokolü, TLS güvenliği, gerçek zamanlı akış
- Bulut Mimarisi: Çoklu servis entegrasyonu, olay güdümlü tasarım
- Veri İşleme: Gerçek zamanlı analitik ve anomali tespiti
- Veritabanı Tasarımı: NoSQL şema tasarımı, bölüm ve sıralama anahtarları
- Sunucusuz Bilgi İşlem: AWS Lambda ile olay tabanlı işleme
- Bulut Güvenliği: IAM rolleri, X.509 sertifikalar, şifreli iletişim

- DevOps: Git sürüm kontrolü

9.2 Sistem Özeti

Başlık	Sonuç
Toplam İşlenen Mesaj	11 mesaj
Başarı Oranı	%100
Anomali Tespiti	Çalışıyor
DynamoDB Depolama	Doğrulandı
Git Commit Sayısı	9 commit

Sistem başarıyla çalışmakta; MQTT üzerinden gerçek zamanlı veri alımı, akıllı veri işleme ve anomali tespiti, DynamoDB'de ölçeklenebilir depolama ve eksiksiz izleme altyapısı sağlanmaktadır.